



Universidad Nacional de La Matanza

Algoritmos y Estructuras de Datos

Trabajo Práctico Laberintos y Fantasma

Comisión 02-3300 (Miércoles 8 a 12hs)

Docentes: Guatelli Renata, Witold Pablo

Grupo: Los predicadores de Linux

Alumnos: Conforti Luis, Gallo Valentina,

Juarez Javier, Porras Federico

2º Cuatrimestre 2025



Instalación.....	2
Descargas.....	2
SDL2.....	2
Información resumida.....	2
Información detallada.....	3
SDL_image.....	10
SDL_ttf.....	11
Preparación del Entorno.....	12
Conceptos Básicos.....	14
Como Jugar.....	14
Ejecutando archivos con archivo .bat (Recomendado, más simple).....	14
Ejecutando archivos a partir de los proyectos.....	14
Interfaz.....	18
Viendo configuraciones y rankings.....	20
Diseño.....	22
Bibliotecas.....	22
Cliente.....	23
Servidor.....	26
Comunicación cliente - servidor.....	27
Funcionalidades añadidas.....	28
Algoritmos.....	29
Algoritmo de Generación de Laberinto.....	29
Pasos principales.....	29
1) Generar la entrada y el primer camino.....	29
2) Generar todos los caminos con el algoritmo DFS.....	29
3) Ajustar los caminos del mapa.....	31
4) Insertar entidades en el mapa.....	32
Algoritmo de Movimiento de Fantasmas.....	35



Instalación

Descargas

En este apartado explicaremos cómo descargar las bibliotecas que utilizamos para el desarrollo del juego. Para aquellos expertos que conocen perfectamente cómo instalar bibliotecas desde github les recomendamos leer la información resumida, con eso será suficiente. Sin embargo, también agregamos información más detallada explicando paso a paso el proceso de instalación.

Para poder comprender bien el proceso de instalación debe leer la instalación de cada biblioteca en el orden dispuesto en este documento, ya que se explica de manera secuencial y no se detalla demasiado cuando los pasos se repiten en las distintas instalaciones, sino que se explica a fondo en la primera y luego se avanza más rápido.

Este instructivo de instalación está desarrollado en base a los archivos que se entregan por mí. Sin embargo ante cualquier inconveniente no dude en consultar el github del proyecto, que utilizamos para trabajar de forma colaborativa y que tiene todos los archivos y configuraciones necesarias realizadas correctamente (a partir del repositorio el proceso de instalación es extremadamente simple).

Repositorio de GitHub: <https://github.com/Juarez0Javier/PhantomDungeon>

Nota: Recuerde instalar todas las bibliotecas, no solo SDL2

SDL2

Información resumida

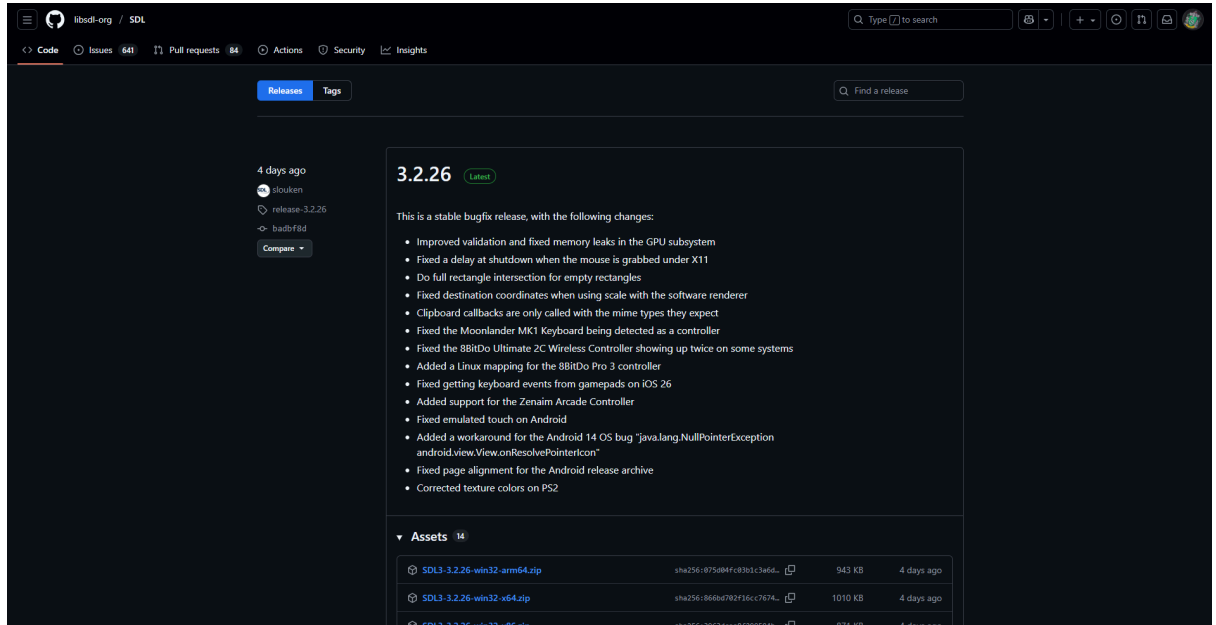
1. En la página <https://github.com/libsdl-org/SDL/releases> busque la versión 2.0.2 (<https://github.com/libsdl-org/SDL/releases?q=2.0.22&expanded=true>), y seleccione la descarga “SDL2-devel-2.0.22-mingw.zip” (<https://github.com/libsdl-org/SDL/releases/download/release-2.0.22/SDL2-devel-2.0.22-mingw.zip>).
2. Copie y descomprima el archivo en la ruta “libs\SDL2_linkNeeded” respecto a la raíz donde tenga descargado el juego (para más claridad ver paso 5 de información detallada)
3. Elija la carpeta correspondiente a su arquitectura (x86 para 64bits, i686 para 32bits), y de ella copie los archivos indicados en la siguiente imagen (en color azul) a la carpeta “SDL2_linkNeeded”, luego ya puede borrar el .zip y la carpeta extraída de él. (Puede ver como le tienen que quedar los archivos en los pasos 9 y 10 de información detallada)

```
D:\FACU\algoritmos\999_TPParcial\Repo\libs\SDL2_linkNeededs\SDL2-2.0.22\x86_64-w64-mingw32>tree
D:
├── bin (Aquí se encuentra el .dll)
├── include
│   └── SDL2 (Aquí se encuentran los .h)
├── lib (Aquí se encuentran los .a y .la)
│   ├── cmake
│   │   └── SDL2
│   └── pkgconfig
├── share
└── aclocal
```

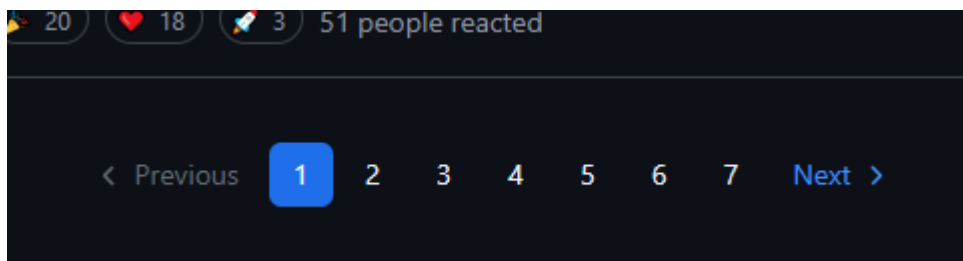


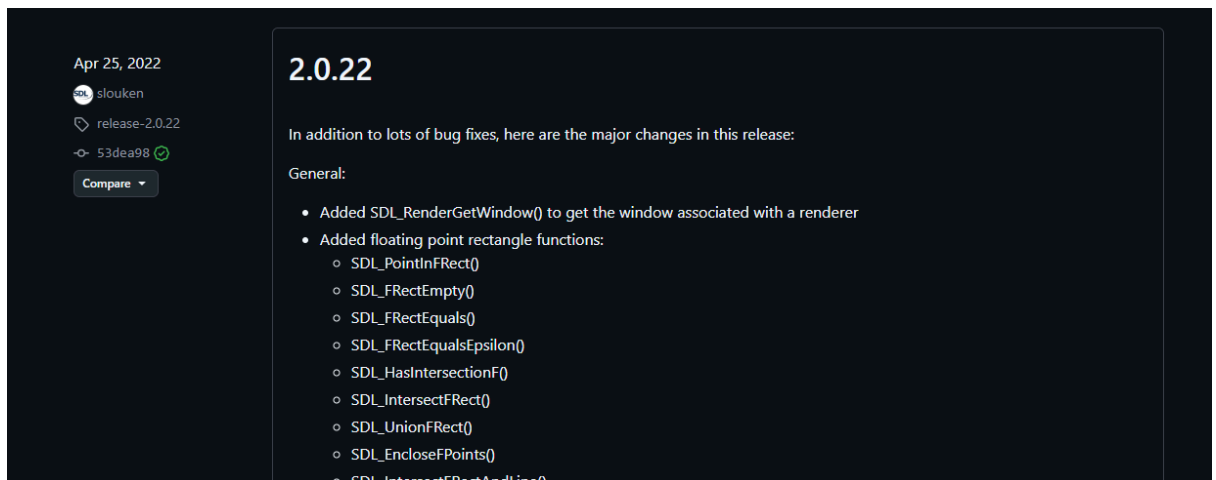
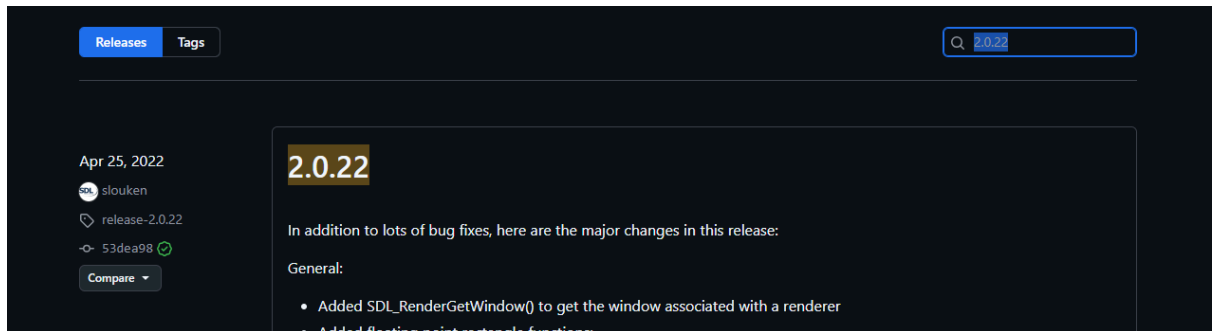
Información detallada

1. En la página <https://github.com/libsdl-org/SDL/releases> encontrará todos los releases de SDL.

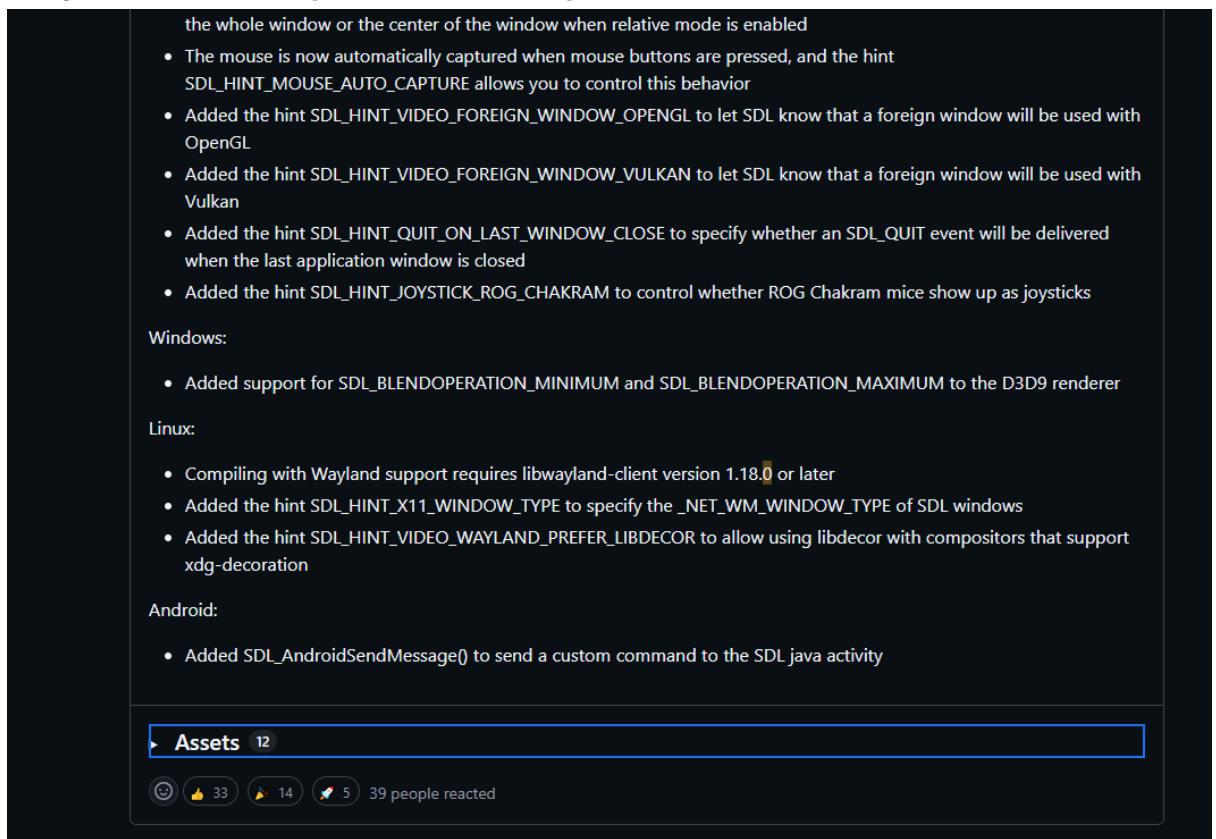


2. Allí deberá buscar la versión 2.0.22, puede hacerlo yendo al final de la página y luego buscándola página por página, o bien ingresar el número de versión en el buscador (más recomendable). Actualmente (*Noviembre de 2025*) está en la página 6.





3. Una vez encontrada, en el final haga clic en “Assets”. Si no la encontró puede utilizar el siguiente link <https://github.com/libsdl-org/SDL/releases?q=2.0.22&expanded=true>



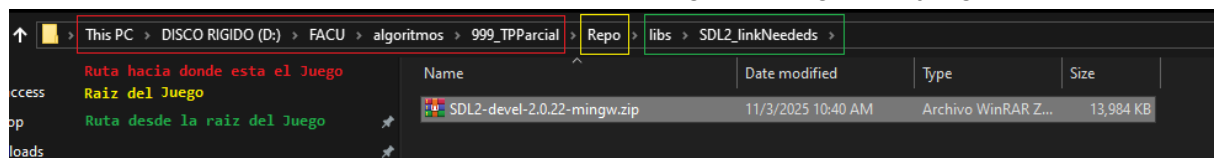
4. Se desplegarán diferentes posibles descargas, de entre ellas debe elegir “SDL2-devel-2.0.22-mingw.zip”. Con ello se descargara un archivo comprimido .zip.



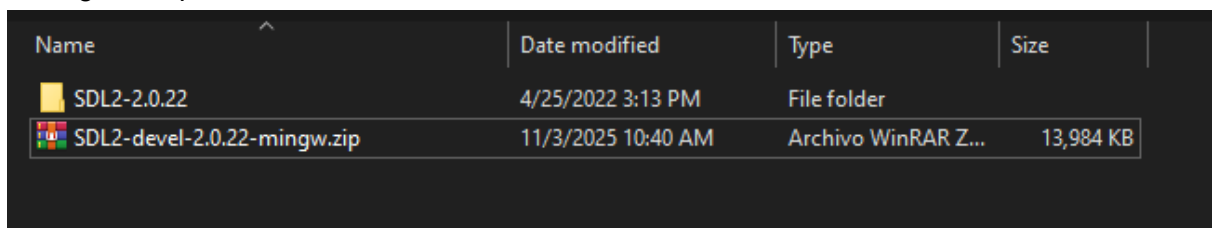
Si no lo encontró puede utilizar el link
<https://github.com/libsdl-org/SDL/releases/download/release-2.0.22/SDL2-devel-2.0.22-mingw.zip>)



- Coloque el .zip descargado en la carpeta SDL2_linkNeededs (en la ruta /libs/SDL2_linkNeededs respecto a la raíz donde tenga descargado el juego)



- Extraiga el .zip





7. Dentro de la carpeta “SDL2-2.0.22” elija la carpeta correspondiente a su arquitectura (en el caso de este instructivo se elegirá x86 para 64bits, para 32bits utilice i686)

algoritmos > 999_TPParcial > Repo > libs > SDL2_linkNeededs > SDL2-2.0.22				
Name	Date modified	Type	Size	
docs	4/25/2022 3:13 PM	File folder		
i686-w64-mingw32	10/31/2018 12:06 PM	File folder		
test	4/25/2022 3:13 PM	File folder		
x86_64-w64-mingw32	10/31/2018 12:06 PM	File folder		
BUGS.txt	4/25/2022 3:13 PM	Documento de te...	1 KB	
COPYING.txt	12/21/2020 2:48 PM	Documento de te...	1 KB	
CREDITS.txt	4/25/2022 3:13 PM	Documento de te...	2 KB	
INSTALL.txt	9/5/2017 2:10 AM	Documento de te...	1 KB	
Makefile	1/29/2018 3:12 AM	File	2 KB	
README.txt	12/21/2020 2:48 PM	Documento de te...	1 KB	
README-SDL.txt	4/25/2022 3:13 PM	Documento de te...	1 KB	
WhatsNew.txt	4/25/2022 3:13 PM	Documento de te...	44 KB	

8. Allí encontrará la estructura de carpetas (para obtener esta visualización utilice el comando tree en la carpeta correspondiente). Los textos en color azul no se encuentran en el output, fueron agregados para dar claridad.

```
D:\FACU\algoritmos\999_TPParcial\Repo\libs\SDL2_linkNeededs\SDL2-2.0.22\x86_64-w64-mingw32>tree
D:..
├── bin (Aquí se encuentra el .dll)
├── include
│   └── SDL2 (Aquí se encuentran los .h)
├── lib (Aquí se encuentran los .a y .la)
├── cmake
│   └── SDL2
├── pkgconfig
├── share
└── aclocal
```

9. Deberá copiar en la carpeta original “SDL2_linkNeededs” los archivos mencionados en azul en el ítem anterior. De la carpeta bin deberá copiar el .dll, de la carpeta include/SDL2 deberá copiar todos los .h, y de la carpeta /lib deberá copiar todos los .a y .la

algoritmos > 999_TPParcial > Repo > libs > SDL2_linkNeededs > SDL2-2.0.22 > x86_64-w64-mingw32 > bin				
Name	Date modified	Type	Size	
SDL2.dll	4/25/2022 3:13 PM	Application exten...	2,203 KB	
sdl2-config	4/25/2022 3:12 PM	File	2 KB	



algoritmos > 999_TPParcial > Repo > libs > SDL2_linkNeeded > SDL2-2.0.22 > x86_64-w64-mingw32 > include > SDL2

begin_code.h	close_code.h	SDL.h	SDL_assert.h
SDL_atomic.h	SDL_audio.h	SDL_bits.h	SDL_blendmode.h
SDL_clipboard.h	SDL_config.h	SDL_cpuinfo.h	SDL_egl.h
SDL_endian.h	SDL_error.h	SDL_events.h	SDL_filesystem.h
SDL_gamecontroller.h	SDL_gesture.h	SDL_haptic.h	SDL_hidapi.h
SDL_hints.h	SDL_joystick.h	SDL_keyboard.h	SDL_keycode.h
SDL_loadso.h	SDL_locale.h	SDL_log.h	SDL_main.h
SDL_messagebox.h	SDL_metal.h	SDL_misc.h	SDL_mouse.h
SDL_mutex.h	SDL_name.h	SDL_opengl.h	SDL_opengl_glext.h
SDL_opengles.h	SDL_opengles2.h	SDL_opengles2_glext.h	SDL_opengles2_glxext.h
SDL_opengles2_glxplatform.h	SDL_opengles2_khrplatform.h	SDL_pixels.h	SDL_platform.h
SDL_power.h	SDL_quit.h	SDL_rect.h	SDL_render.h
SDL_revision.h	SDL_rwops.h	SDL_scancode.h	SDL_sensor.h
SDL_shape.h	SDL_stdinc.h	SDL_surface.h	SDL_system.h
SDL_syswm.h	SDL_test.h	SDL_test_assert.h	SDL_test_common.h
SDL_test_compare.h	SDL_test_crc32.h	SDL_test_font.h	SDL_test_fuzzer.h
SDL_test_harness.h	SDL_test_images.h	SDL_test_log.h	SDL_test_md5.h
SDL_test_memory.h	SDL_test_random.h	SDL_thread.h	SDL_timer.h
SDL_touch.h	SDL_types.h	SDL_version.h	SDL_video.h
SDL_vulkan.h			

algoritmos > 999_TPParcial > Repo > libs > SDL2_linkNeeded > SDL2-2.0.22 > x86_64-w64-mingw32 > lib >

Name	Date modified	Type	Size
cmake	10/31/2018 12:06 PM	File folder	
pkgconfig	4/25/2022 3:12 PM	File folder	
libSDL2.a	4/25/2022 3:13 PM	A File	19,881 KB
libSDL2.dll.a	4/25/2022 3:13 PM	A File	503 KB
libSDL2.la	4/25/2022 3:13 PM	LA File	2 KB
libSDL2_test.a	4/25/2022 3:13 PM	A File	611 KB
libSDL2_test.la	4/25/2022 3:13 PM	LA File	1 KB
libSDL2main.a	4/25/2022 3:13 PM	A File	10 KB
libSDL2main.la	4/25/2022 3:13 PM	LA File	1 KB

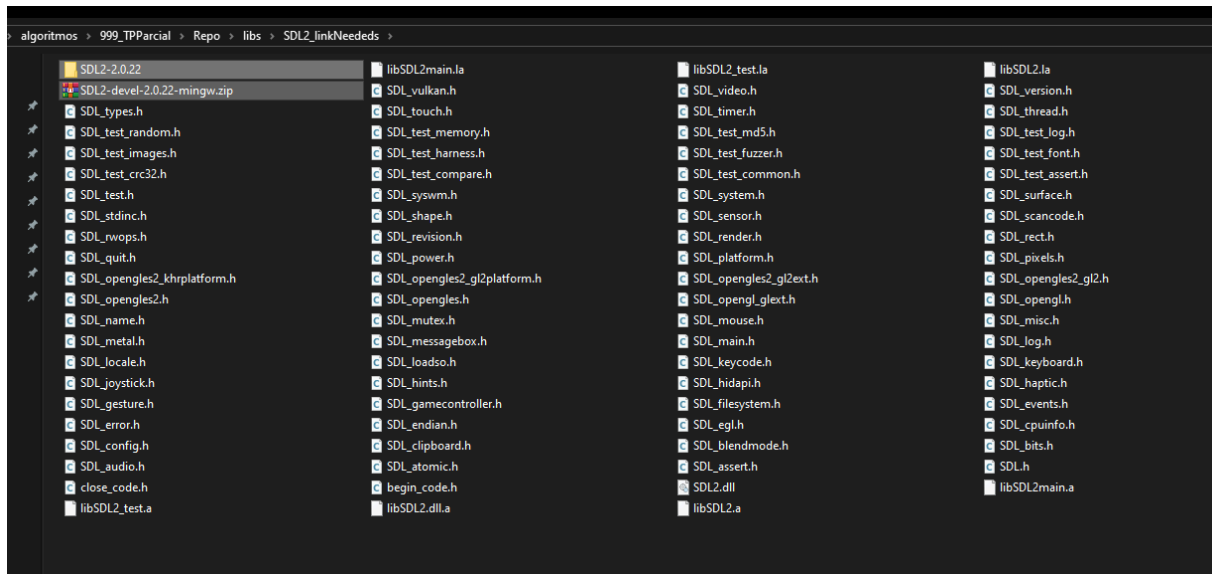
Quedando de la siguiente manera

algoritmos > 999_TPParcial > Repo > libs > SDL2_linkNeeded

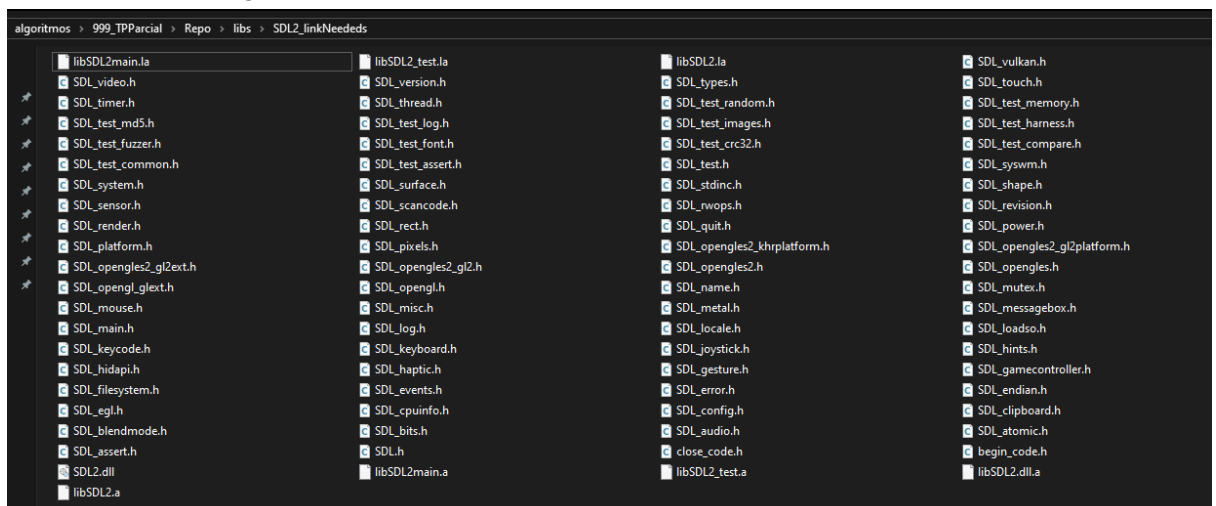
SDL2-2.0.22	libSDL2main.la	libSDL2_test.la	libSDL2.la
SDL2-devel-2.0.22-mingw.zip	SDL_vulkan.h	SDL_video.h	SDL_version.h
SDL_types.h	SDL_touch.h	SDL_timer.h	SDL_thread.h
SDL_test_random.h	SDL_test_memory.h	SDL_test_md5.h	SDL_test_log.h
SDL_test_images.h	SDL_test_harness.h	SDL_test_fuzzer.h	SDL_test_font.h
SDL_test_crc32.h	SDL_test_compare.h	SDL_test_common.h	SDL_test_assert.h
SDL_test.h	SDL_syswm.h	SDL_system.h	SDL_surface.h
SDL_stdinc.h	SDL_shape.h	SDL_sensor.h	SDL_scancode.h
SDL_rwops.h	SDL_revision.h	SDL_renderer.h	SDL_rect.h
SDL_quit.h	SDL_power.h	SDL_platform.h	SDL_pixels.h
SDL_opengles2_khrplatform.h	SDL_opengles2_glxplatform.h	SDL_opengles2_glxext.h	SDL_opengles2_glext.h
SDL_opengles2.h	SDL_opengles.h	SDL_opengl_glext.h	SDL_opengl.h
SDL_name.h	SDL_mutex.h	SDL_mouse.h	SDL_misc.h
SDL_metal.h	SDL_messagebox.h	SDL_main.h	SDL_log.h
SDL_locale.h	SDL_loadso.h	SDL_keycode.h	SDL_keyboard.h
SDL_joystick.h	SDL_hints.h	SDL_hidapi.h	SDL_haptic.h
SDL_gesture.h	SDL_gamecontroller.h	SDL_filesystem.h	SDL_events.h
SDL_error.h	SDL_endian.h	SDL_egl.h	SDL_cpuinfo.h
SDL_config.h	SDL_clipboard.h	SDL_blendmode.h	SDL_bits.h
SDL_audio.h	SDL_atomic.h	SDL_assert.h	SDL.h
close_code.h	begin_code.h	SDL2.dll	libSDL2main.a
libSDL2_test.a	libSDL2.dll.a	libSDL2.a	



10. Ahora ya puede eliminar el .zip y la carpeta de donde fueron sacados los archivos



Quedando de la siguiente manera:



```
D:\FACU\algoritmos\999_TPParcial\Repo\libs\SDL2_linkNeeded>tree /f
D:.
```

```
begin_code.h
close_code.h
libSDL2.a
libSDL2.dll.a
libSDL2.la
libSDL2main.a
libSDL2main.la
libSDL2_test.a
libSDL2_test.la
SDL.h
SDL2.dll
SDL_assert.h
SDL_atomic.h
SDL_audio.h
SDL_bits.h
SDL_blendmode.h
SDL_clipboard.h
SDL_config.h
SDL_cpuinfo.h
SDL_egl.h
SDL_endian.h
SDL_error.h
SDL_events.h
```



```
SDL_filesystem.h
SDL_gamecontroller.h
SDL_gesture.h
SDL_haptic.h
SDL_hidapi.h
SDL_hints.h
SDL_joystick.h
SDL_keyboard.h
SDL_keycode.h
SDL_loadso.h
SDL_locale.h
SDL_log.h
SDL_main.h
SDL_messagebox.h
SDL_metal.h
SDL_misc.h
SDL_mouse.h
SDL_mutex.h
SDL_name.h
SDL_opengl.h
SDL_opengles.h
SDL_opengles2.h
SDL_opengles2_glx.h
SDL_opengles2_glxext.h
SDL_opengles2_khrplatform.h
SDL_opengl_glxext.h
SDL_pixels.h
SDL_platform.h
SDL_power.h
SDL_quit.h
SDL_rect.h
SDL_render.h
SDL_revision.h
SDL_rwops.h
SDL_scancode.h
SDL_sensor.h
SDL_shape.h
SDL_stdinc.h
SDL_surface.h
SDL_system.h
SDL_syswm.h
SDL_test.h
SDL_test_assert.h
SDL_test_common.h
SDL_test_compare.h
SDL_test_crc32.h
SDL_test_font.h
SDL_test_fuzzer.h
SDL_test_harness.h
SDL_test_images.h
SDL_test_log.h
SDL_test_md5.h
SDL_test_memory.h
SDL_test_random.h
SDL_thread.h
SDL_timer.h
SDL_touch.h
SDL_types.h
SDL_version.h
SDL_video.h
SDL_vulkan.h
```

No subfolders exist



SDL_image

1. Descargue la biblioteca (análogo a la descarga de SDL2) en su versión 2.8.8 la descarga con nombre "SDL2_image-devel-2.8.8-mingw.zip". Los links se adjuntan a continuación:

Releases: https://github.com/libsdl-org/SDL_image/releases
 Release 2.8.8: https://github.com/libsdl-org/SDL_image/releases/tag/release-2.8.8
 Descarga directa ("SDL2_image-devel-2.8.8-mingw.zip"):
https://github.com/libsdl-org/SDL_image/releases/download/release-2.8.8/SDL2_image-devel-2.8.8-mingw.zip

2. Copie el .zip a la carpeta "libs\SDL2_more" y descomprimalo allí
3. Elija la carpeta correspondiente a su arquitectura (x86 para 64bits, i686 para 32bits), y de ella copie los archivos indicados en la siguiente imagen (en color azul) a la carpeta "SDL2_more", luego ya puede borrar el .zip y la carpeta extraída de él.

```
D:\FACU\algoritmos\999_TPParcial\Repo\libs\SDL2_more\SDL2_image-2.8.8\x86_64-w64-mingw32>tree
D: .
├── bin (Aquí se encuentra el .dll)
├── include (Aquí se encuentra el .h)
├── SDL2
├── lib
├── cmake
├──   └── SDL2_image
└── pkgconfig
```

Quedandole de la siguiente manera:

algoritmos > 999_TPParcial > Repo > libs > SDL2_more				
Name	Date modified	Type	Size	
 SDL_image.h	3/3/2025 9:53 PM	Archivo de origen ...	70 KB	
 SDL2_image.dll	3/3/2025 9:53 PM	Application exten...	169 KB	



SDL_ttf

1. Descargue la biblioteca (análogo a la descarga de SDL2 Image) en su versión 2.24.0 la descarga con nombre "SDL2_ttf-devel-2.24.0-mingw.zip". Los links se adjuntan a continuación:

Releases: https://github.com/libsdl-org/SDL_ttf/releases
 Release 2.8.8: https://github.com/libsdl-org/SDL_ttf/releases/tag/release-2.24.0
 Descarga directa ("SDL2_image-devel-2.8.8-mingw.zip"):
https://github.com/libsdl-org/SDL_ttf/releases/download/release-2.24.0/SDL2_ttf-devel-2.24.0-mingw.zip

2. Copie el .zip a la carpeta "libs\SDL2_more" y descomprimalo allí
3. Elija la carpeta correspondiente a su arquitectura (x86 para 64bits, i686 para 32bits), y de ella copie los archivos indicados en la siguiente imagen (en color azul) a la carpeta "SDL2_more", luego ya puede borrar el .zip y la carpeta extraída de él.

```
D:\FACU\algoritmos\999_TPParcial\Repo\libs\SDL2_more\SDL2_ttf-2.24.0\x86_64-w64-mingw32>tree
D:..
├── bin (Aquí se encuentra el .dll)
├── include
│   └── SDL2 (Aquí se encuentra el .h)
├── lib
│   ├── cmake
│   │   └── SDL2_ttf
│   └── pkgconfig
```

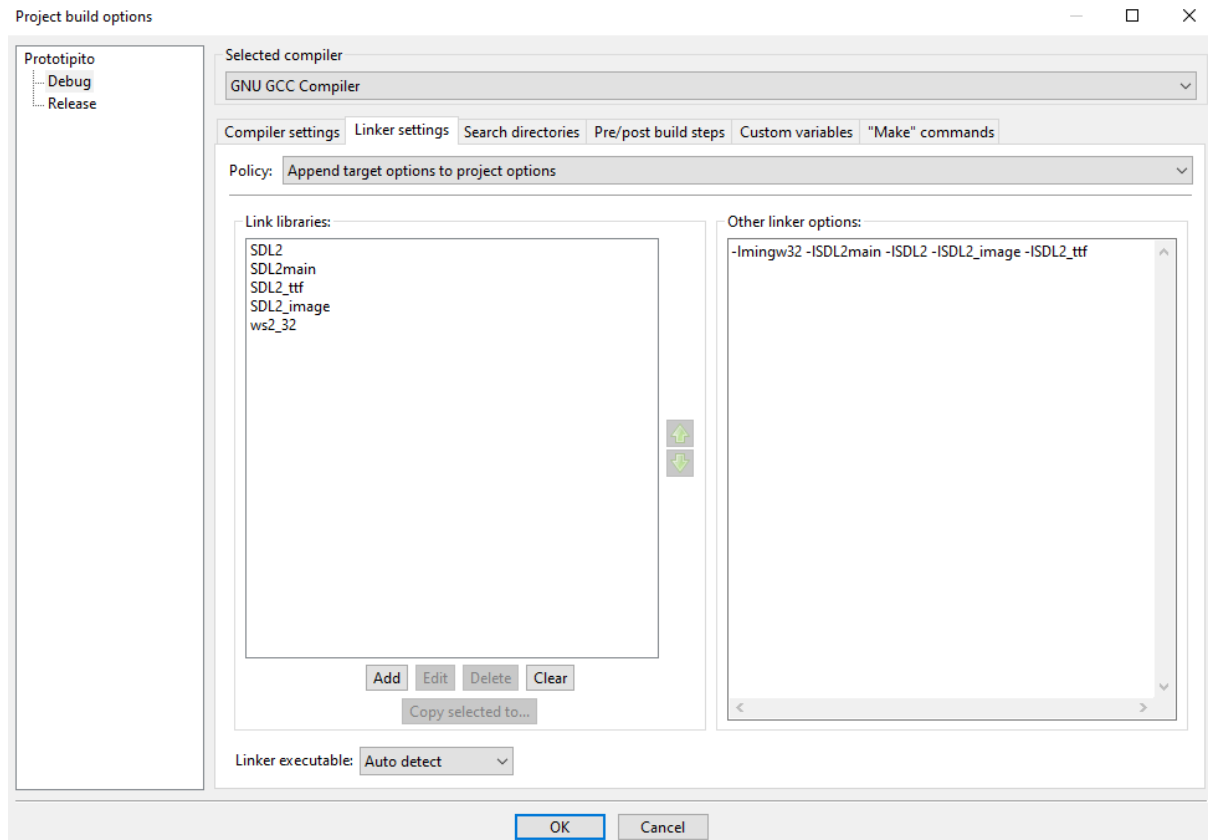
Quedándole de la siguiente manera:

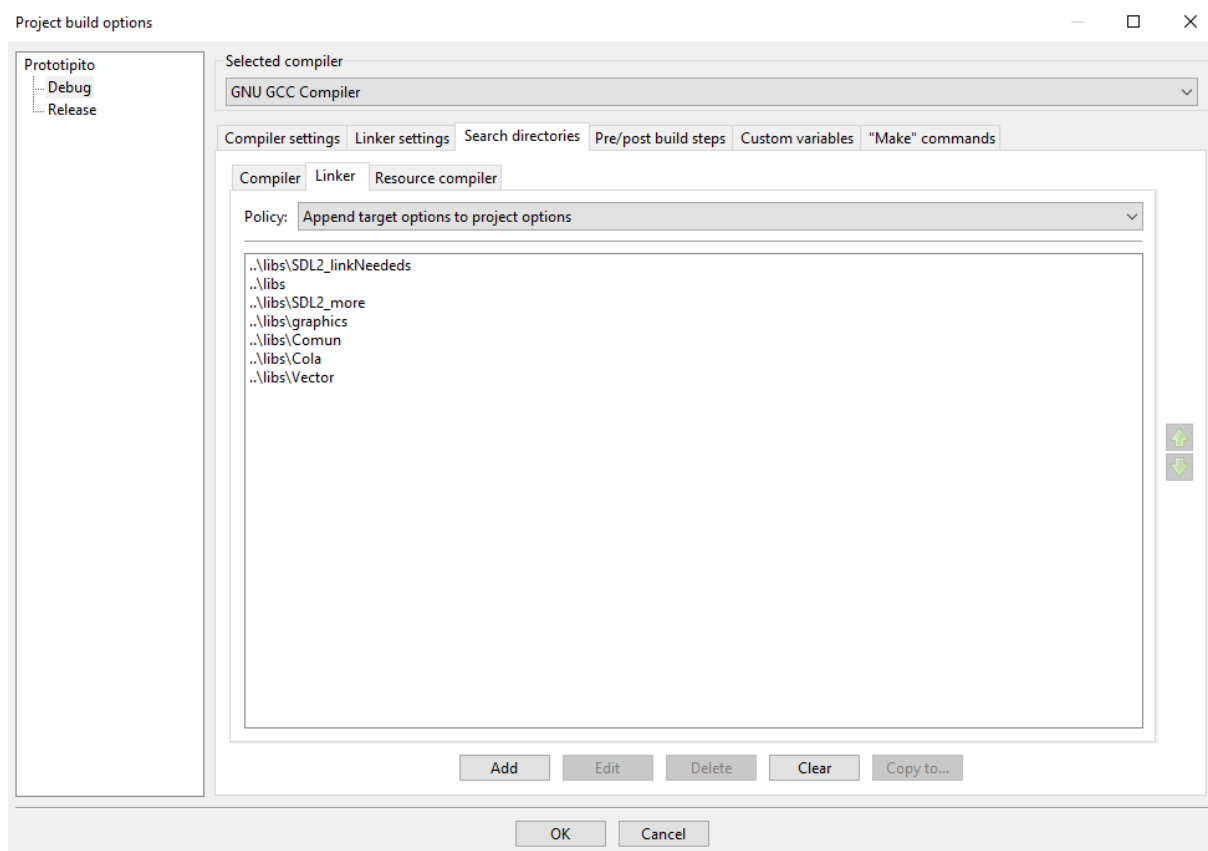
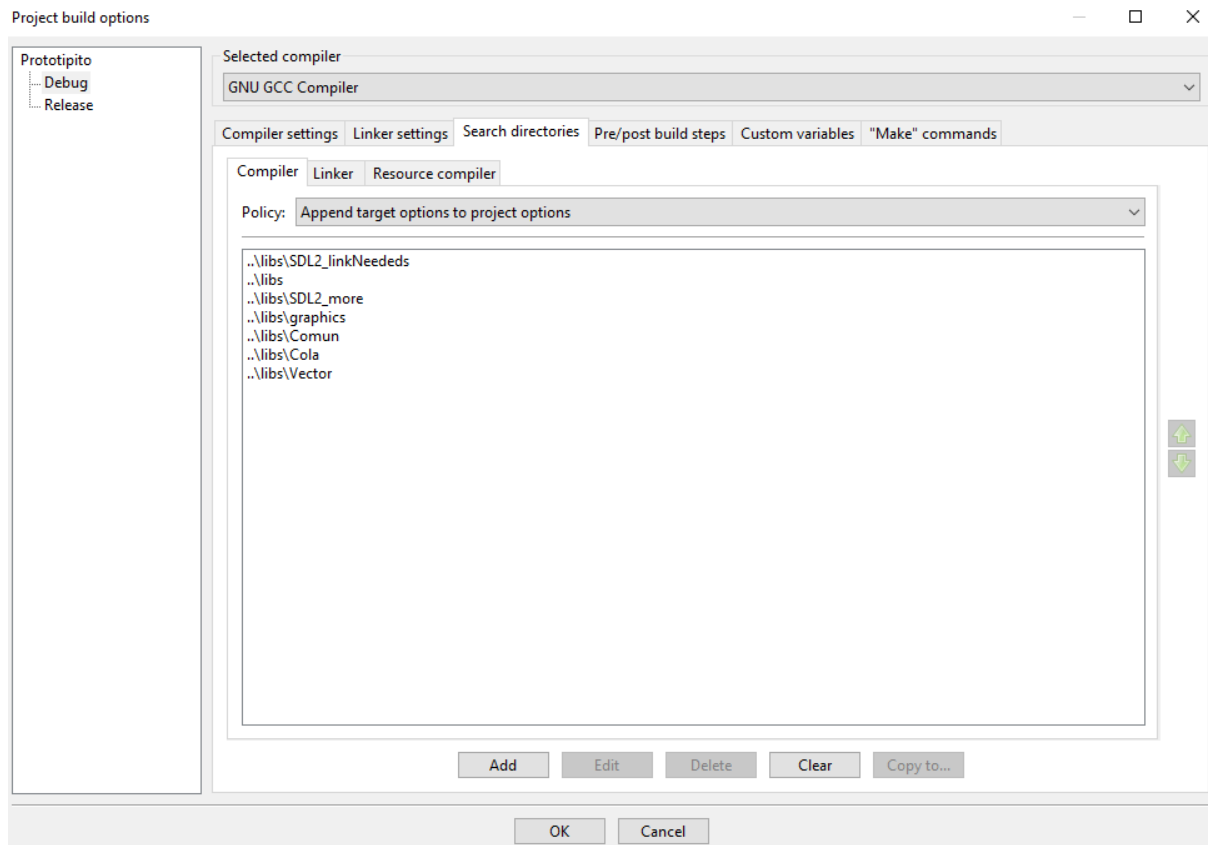
U > algoritmos > 999_TPParcial > Repo > libs > SDL2_more				
	Name	Date modified	Type	Size
✦	SDL_image.h	3/3/2025 9:53 PM	Archivo de origen ...	70 KB
✦	SDL_ttf.h	1/2/2025 3:46 PM	Archivo de origen ...	87 KB
✦	SDL2_image.dll	3/3/2025 9:53 PM	Application exten...	169 KB
✦	SDL2_ttf.dll	1/2/2025 3:46 PM	Application exten...	1,766 KB



Preparación del Entorno

Se adjuntará un proyecto de codeblocks para ahorrarle tener que linkear todo, sin embargo dejamos aquí algunas imágenes de como deben quedar los linkeos en el proyecto.





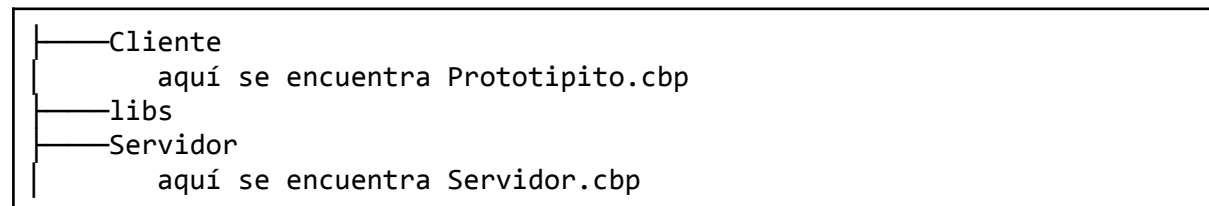
Teniendo esa estructura ya armada compile el proyecto.
Una vez compilado desde codeblocks ya puede ejecutarlo si lo desea.



Conceptos Básicos

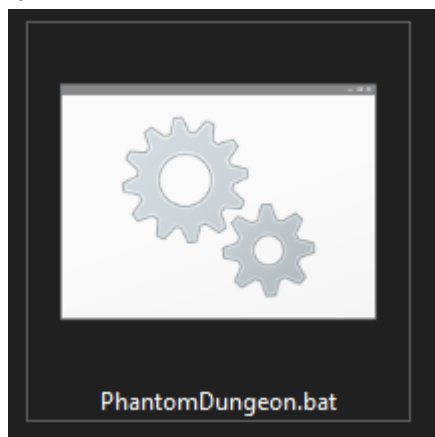
Como Jugar

Para poder jugar primero debe compilar tanto el proyecto del Cliente como del Servidor. Recuerde que los .cbp se encuentran en las carpetas “./Servidor” y “./Cliente” respecto a la raíz



Ejecutando archivos con archivo .bat (Recomendado, más simple)

Creamos un archivo .bat que realiza las ejecuciones necesarias de modo que con solo ejecutar el .bat es suficiente para jugar el juego.



De todos modos adjuntamos posteriormente como ejecutar desde los proyectos de codeblocks.

Ejecutando archivos a partir de los proyectos

Atención: Si desea optar por esta forma de ejecutar, deberá, en el proyecto que desee (en su archivo de constantes) modificar la macro COMPILACION_DESDE_BIN, dejando su valor en false.

Para el cliente (Cliente/constants.h):



```
50 // Rutas
51 #define COMPILACION_DESDE_BIN false
52 #if COMPILACION_DESDE_BIN
53
54 // Para cuando se ejecuta del binario.
55 #define RUTA_LABERINTO_PRESET "../..laberintos/laberinto.txt"
56 #define RUTA_CONFIGS "../..config.txt"
57 #define RUTA_FONDO "../..src/img/background.png"
58 #define RUTA_IMG "../..src/img/"
59 #define RUTA_BOTONES "../..src/img/botones.png"
60 #define RUTA_FUENTE "../..fnt/Consolas-Regular.ttf"
61
62 #else
63
64 // Para cuando se ejecuta de codeblocks.
65 #define RUTA_LABERINTO_PRESET "../laberintos/laberinto.txt"
66 #define RUTA_CONFIGS "../config.txt"
67 #define RUTA_FONDO "../src/img/background.png"
68 #define RUTA_IMG "../src/img/"
69 #define RUTA_BOTONES "../src/img/botones.png"
70 #define RUTA_FUENTE "../fnt/Consolas-Regular.ttf"
71
72 #endif
```

Para el servidor (Servidor/constantes.h):

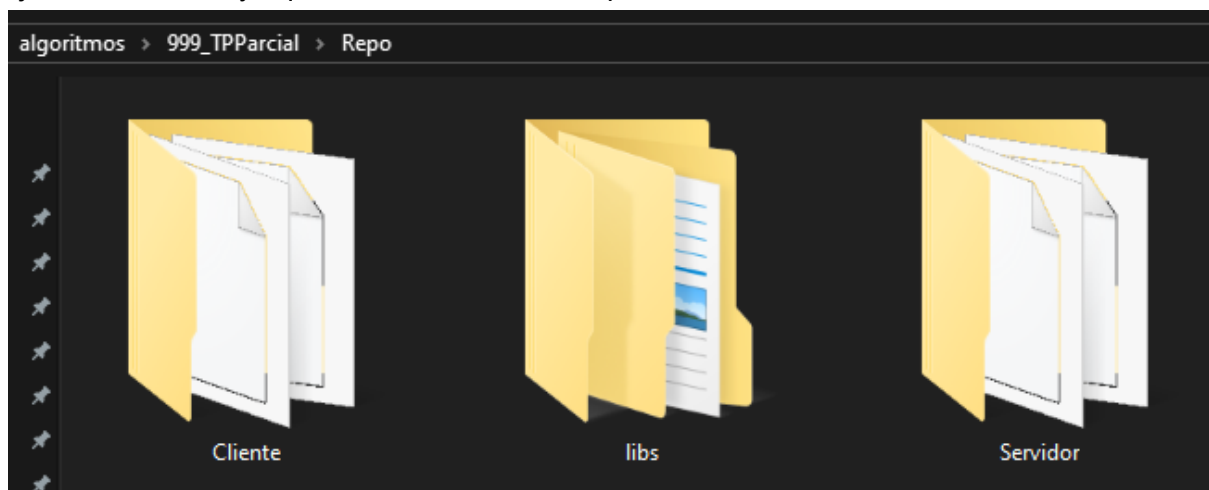


```

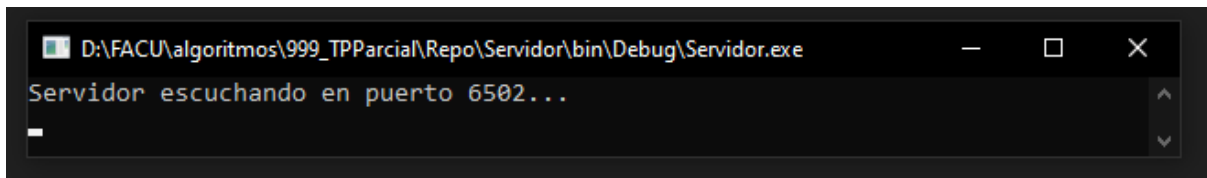
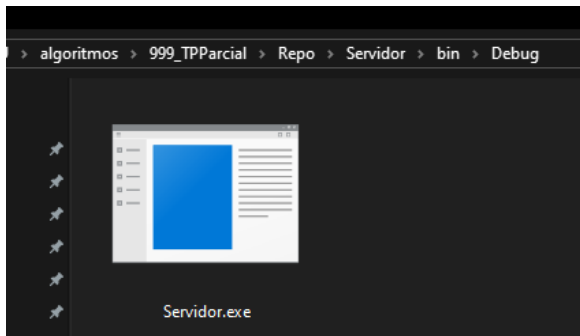
15 // Rutas de archivos
16 #define COMPILACION_DESDE_BIN false
17 #if COMPILACION_DESDE_BIN
18
19 // Cuando se lo ejecuta desde el binario.
20 #define RUTA_JUGADORES_DAT "../..../persistencia/jugadores.dat"
21 #define RUTA_JUGADORES_IDX_DAT "../..../persistencia/jugadoresIdx.dat"
22 #define RUTA_PARTIDAS_DAT "../..../persistencia/partidas.dat"
23 #define RUTA_RANKINGS_DAT "../..../persistencia/rankings.dat"
24
25 #else
26
27 // Cuando se lo ejecuta desde codeblocks.
28 #define RUTA_JUGADORES_DAT "../persistencia/jugadores.dat"
29 #define RUTA_JUGADORES_IDX_DAT "../persistencia/jugadoresIdx.dat"
30 #define RUTA_PARTIDAS_DAT "../persistencia/partidas.dat"
31 #define RUTA_RANKINGS_DAT "../persistencia/rankings.dat"
32
33 #endif
34
35 #endif // CONSTANTES_H_INCLUDED
36

```

El juego está dividido en 2 proyectos de codeblocks, uno para el servidor y otro para el cliente, deberá ejecutar primero el del servidor y luego el del cliente. Recuerde que para poder hacer uso de las funcionalidades del servidor recuerde tenerlo corriendo mientras ejecuta el cliente, ya que caso contrario no se podrá establecer la conexión.



Para poder correr el servidor deberá ejecutar el archivo "Servidor.exe" que se encuentra en la ruta "raizJuego\Servidor\bin\Debug". Tras ello se abrirá una pantalla informando que el servidor ya se encuentra escuchando en el puerto.

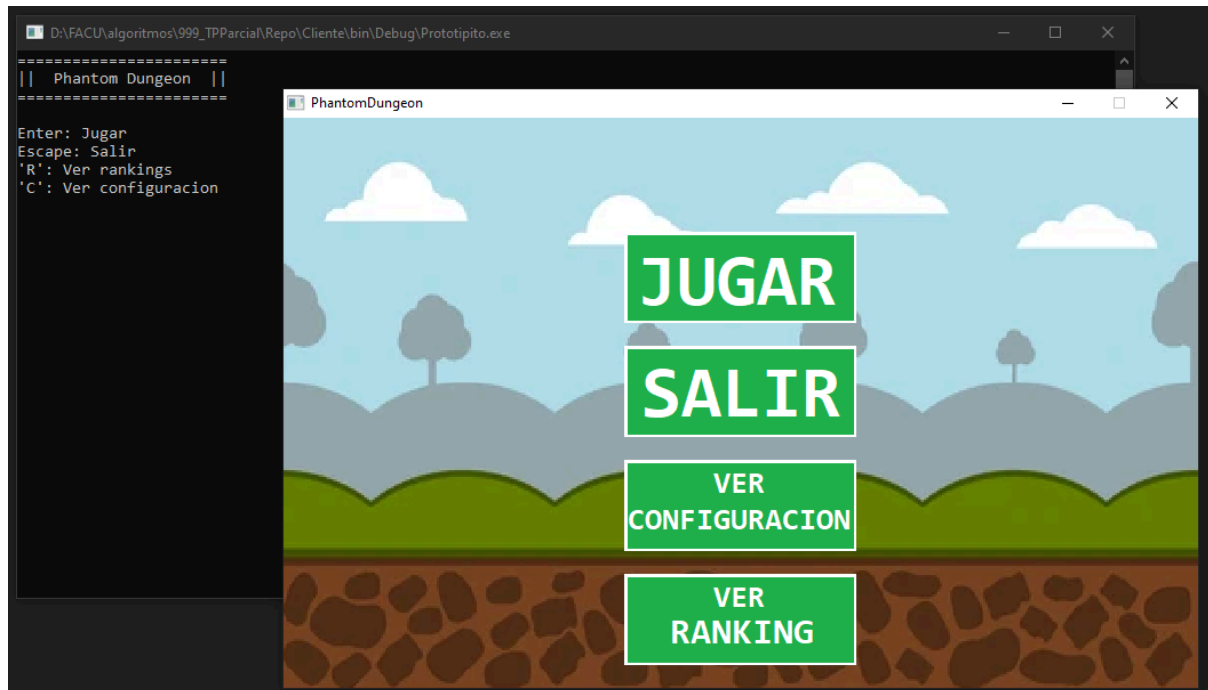




Interfaz

Para poder jugar, ejecute desde codeblocks el proyecto. Se abrirán 2 ventanas, una de consola y otra de interfaz gráfica con SDL.

En el menú de inicio usted tendrá 4 opciones: Jugar, salir, ver configuración y ver ranking.

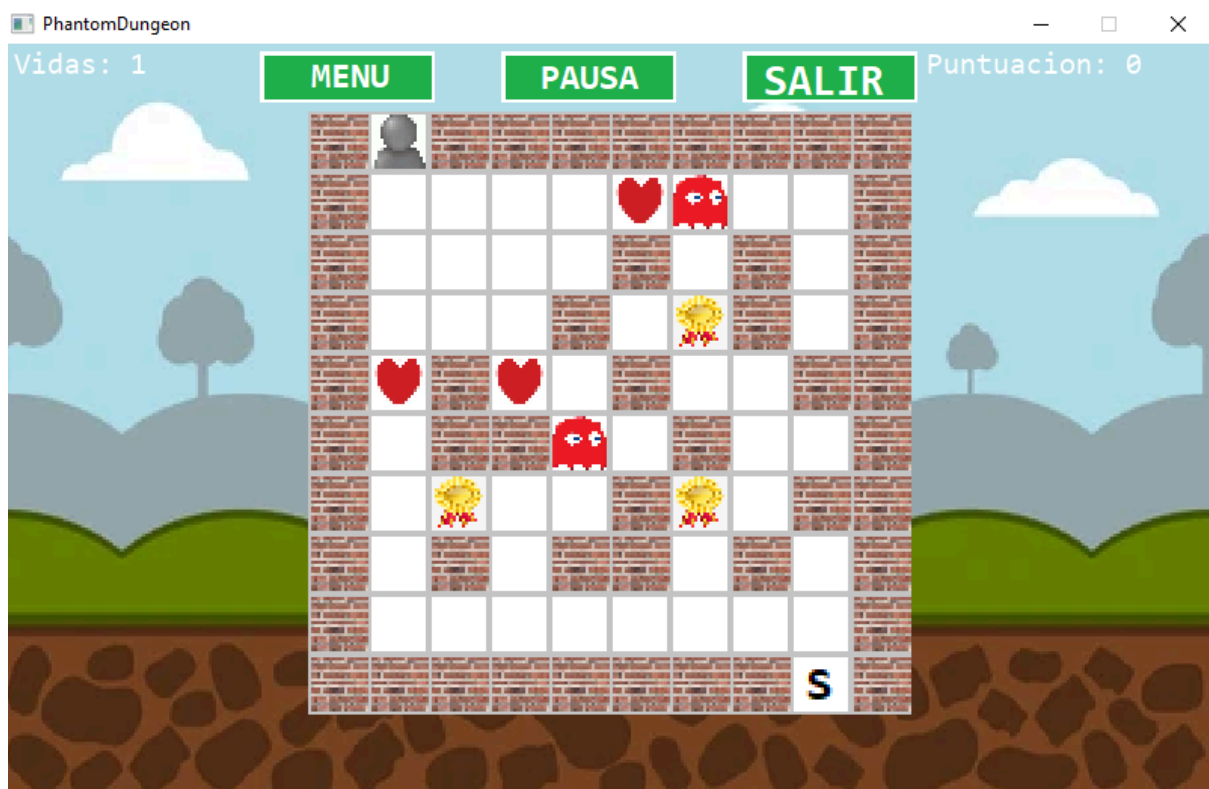


Recuerde que para que el juego pueda recibir los inputs de teclado que usted realice (presionar teclas o ingresar textos) la ventana de SDL deberá estar activa (en primer plano).

Una vez presione jugar, si la conexión con el servidor fue exitosa, se mostrará una pantalla donde podrá ingresar su nombre de jugador para poder acumular su puntuación de distintas partidas en el ranking.



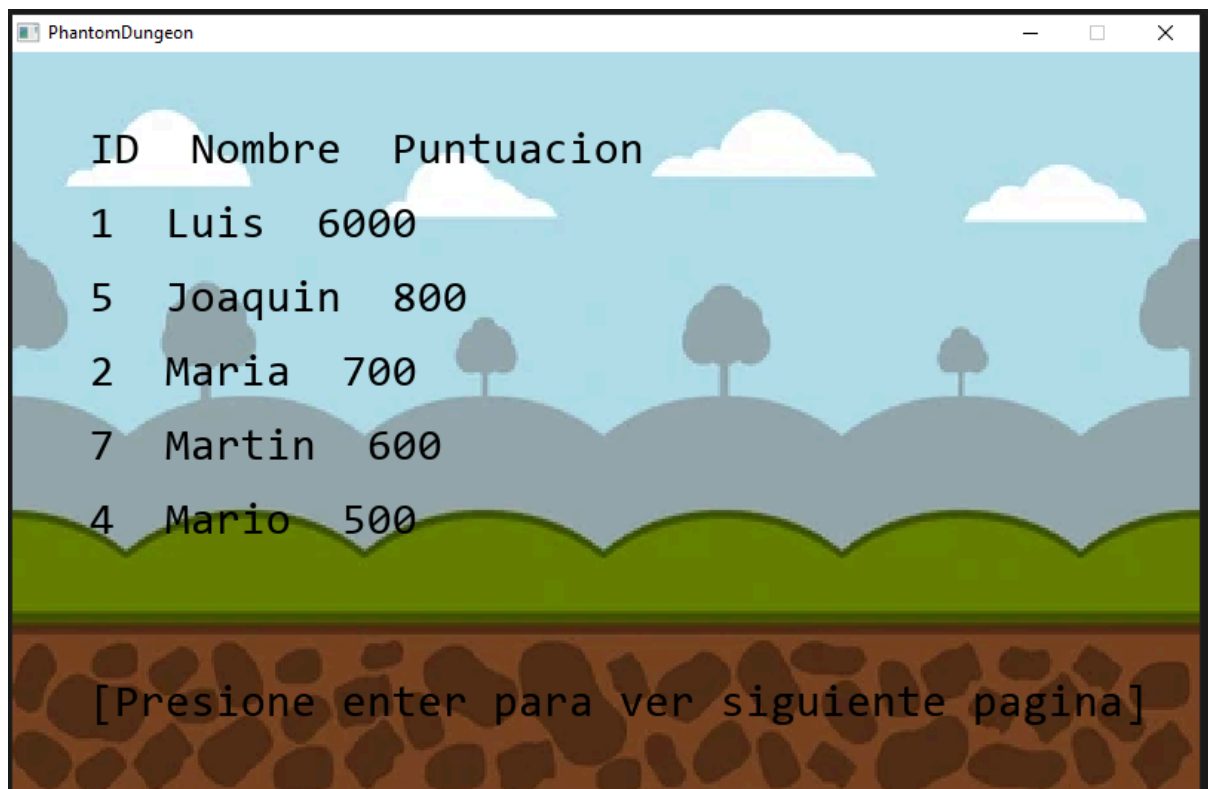
Una vez inicie la partida, podrá ver el laberinto, con los distintos premios, vidas y fantasmas, además de la cantidad de vidas y puntuación que acumulo en esta partida. Una vez llegado este punto ya está listo para jugar y divertirse. Para moverse en el laberinto utilice las flechas del teclado, o bien W,A,S,D.

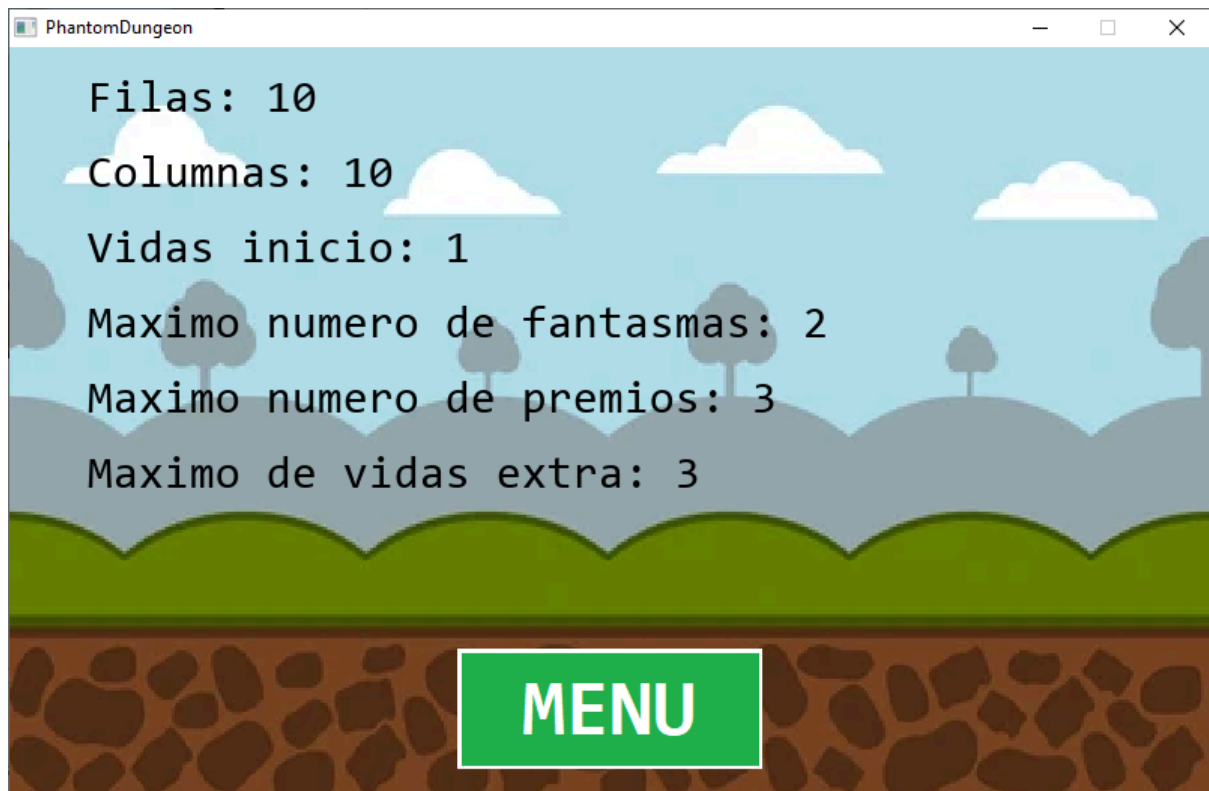




Viendo configuraciones y rankings

Al entrar al juego hay 2 botones para estas funcionalidades, uno que permite ver la configuración preestablecida, y otro que hace lo propio con los rankings.



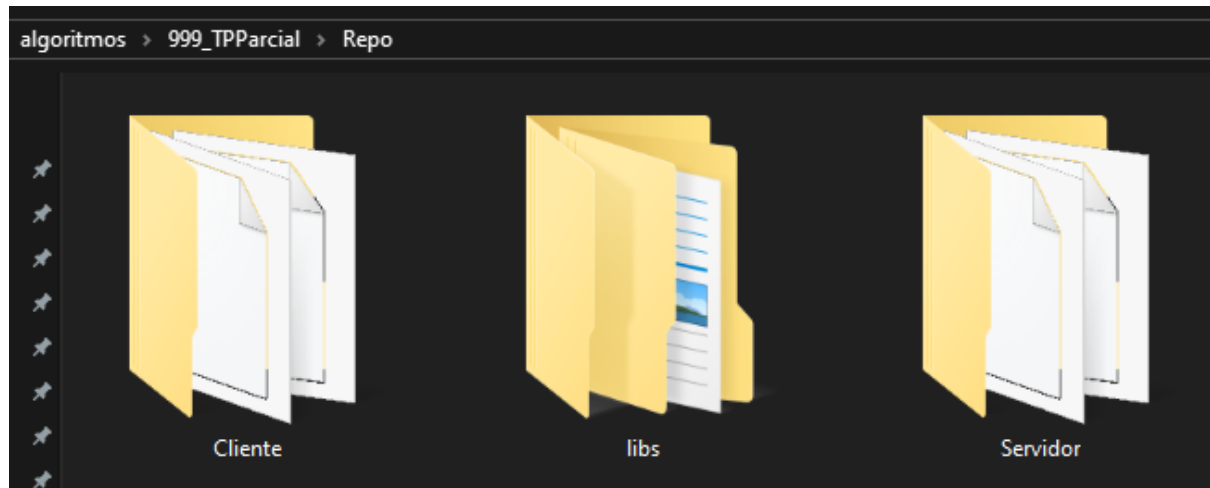




Diseño

El juego está modularizado en distintas carpetas, archivos y funciones para dar claridad y una correcta legibilidad. Además posee algunos archivos que no son de código pero son necesarios para jugarlo.

Como mencionamos anteriormente, el código está dividido Cliente, Servidor y Bibliotecas



Bibliotecas

A continuación una esquematización de las bibliotecas

—SDL2_linkNeededs —SDL2_more	Archivos propios de SDL. Externa.
—graphics	Simplifica el manejo de SDL
—conexion	Realiza conexión Cliente Servidor
—archBin —string —matrizDinamica	Usos Varios
—Pila —Lista —Cola —Arbol —Vector	TDAs
—Comun	Constantes globales
<i>(El orden alfabético fue modificado)</i>	



Cliente

A continuación adjuntamos un recorte de la estructura de carpetas obtenida con `tree /f` en la carpeta “Cliente” con algunas aclaraciones en los casos de archivos que no corresponden a código.

```

config.txt (**)
constants.h
global.h
main.c
main.h
Prototipito.cbp (*)
bin (*)
conexion
  api.c
  api.h
  conexion.c
  conexion.h
  protocolo.c
  protocolo.h
  transporte.c
  transporte.h
configs
  configs.c
  configs.h
entidades
  entidades.c
  entidades.h
fnt (**)
inicios
  inicios.c
  inicios.h
laberintos (**)
mapa
  mapa.c
  mapa.h
  mapaRandom.c
  mapaRandom.h
movimiento
  movimiento.c
  movimiento.h
obj (*)
otros
  otrasFunciones.c
  otrasFunciones.h
partida
  partida.c
  partida.h
secciones
  secciones.c
  secciones.h
src (**)

```

(*) Relacionado a compilación

(**) Relacionado a impresión por interfaz SDL (Imágenes y textos)

(***) Archivos de entrada y salida del programa



Una vez fuera de análisis los archivos que no contienen código, queda lo siguiente:

constants.h	Constantes Globales
global.h	Estructura de Contexto Global tContextoGlobal
main.c (.c y .h)	Se hace un llamado a GHP_SetWindow que genera la interfaz SDL y se le pasa por parámetro react donde estará el código del juego (*)
<pre> ├──conexion │ api (.c y .h) │ conexion (.c y .h) │ protocolo (.c y .h) </pre>	Lógica de conexión y manejo de formato para la comunicación con el servidor.
├──configs (.c y .h)	Carga de configuración. Validación y regeneración en caso de carga inválida.
├──entidades (.c y .h)	Estructura y funciones básicas de entidades
├──inicios (.c y .h)	Inicio de estructura partida, y elementos gráficos como textos, botones y texturas
<pre> ├──mapa │ mapa (.c y .h) │ mapaRandom (.c y .h) </pre>	Estructura y funciones básicas de mapas Generación de laberintos aleatorios
├──movimiento (.c y .h)	Estructura de movimiento, resolver movimientos, calcular movimientos fantasmas
├──otros (.c y .h)	Funciones extra no categorizadas
├──partida (.c y .h)	Estructura Partida
├──secciones (.c y .h)	Código referido a cada sección del juego (*)
(*) El flujo principal del programa pasa por estos archivos	

El funcionamiento de nuestro programa se basa en secciones, cada una de ellas tiene una función init (inicializaciones), una handler (actuador) y algunas de ellas tienen una render (actualizador del renderizado en interfaz). De esta manera cada sección se encuentra en el main dentro del array secciones con sus punteros a función.

A continuación se adjunta un pseudocódigo de cómo funciona el game loop principal. Figura en color azul lo relacionado a el control de frames (necesario para garantizar que el programa corra fluido de igual manera en computadoras con distintos recursos), en verde aquello relacionado directamente a SDL (renderización y captura de eventos) y en violeta lo relacionado a las secciones. (Atención: El siguiente pseudocódigo es solo una aproximación, algunas cosas fueron recortadas y reordenadas para facilitar la comprensión).



```

Seccion secciones[] -> array con punteros a función

seccion = SECCION_MENU; -> seccion con la que se inicia
seccionPrev = seccion; -> variable auxiliar para detectar cambio de sección

if (iniciarPartida) {
    secciones[seccion].init()
    SDLrenderizar()
} else
    seccion = SECCION_SALIR_DIRECTO;

inicializaciónControlFrames();

while (*seccion != SECCION_SALIR_DIRECTO) {

    funcionesControlFrames();

    SDL_CapturarEvento(&event);

    else {
        secciones[seccion].handler(eventoCapturado, &seccion);

        if (*seccion != SECCION_SALIR_DIRECTO && seccionPrev != seccion) {

            seccionPrev = seccion;

            SDLDescartarEventosEncolados();

            // Al cambiar de sección, se inicia la nueva sección.
            secciones[*seccion].init();
            SDLrenderizar(); // algunos init renderizan cosas
        }
    }

    // algunas secciones no tienen render
    if (seccion != SECCION_SALIR_DIRECTO && secciones[seccion].render) {
        secciones[seccion].render();
        SDLrenderizar();
    }
    limitadorDeFrames();
}

```

Dentro de cada handler, se puede cambiar la sección y con ello al volver al loop principal se detecta el cambio descartando todos los eventos que SDL recibió y encoló mientras se estaba en la anterior sección. Tras cambiar de sección, también se inicia y renderiza la nueva sección. De esta manera se puede navegar entre distintas secciones.

El código referido a que se muestra y cuál es el procesamiento de las secciones se encuentra en secciones (.c y .h). Allí se encuentran definidas e implementadas las funciones antes mencionadas (init, handler, render). Además, también se tiene una función handleButtonsClick que ayuda con el manejo de capturar cuando se oprimen botones y cambiar de secciones.



Servidor

El proyecto servidor, de la misma forma que el cliente, se encuentra compuesto de una serie de módulos funcionales para su correcta ejecución. Replicando lo expuesto anteriormente, a continuación se provee de la estructura de carpetas, obtenida con `tree /f` para la carpeta “Servidor”, con aclaraciones sobre algunos archivos y directorios que no poseen código:

```

|   conexion.c
|   conexion.h
|   constantes.h
|   main.c
|   peticiones.c
|   peticiones.h
|   Servidor.cbp (*)
|   bin (*)
|   controladores
|       comun.c
|       comun.h
|       jugador.c
|       jugador.h
|       partida.c
|       partida.h
|       ranking.c
|       ranking.h
|   obj (*)
|   persistencia (**)
|
| (*) Relacionado a compilación
| (**) Contiene los archivos binarios para el guardado de datos en disco.

```

Una vez fuera de análisis los archivos que no contienen código, queda lo siguiente:

conexion (.c y .h)	Realiza la lógica para conectar al servidor, configurando los sockets
constantes.h	Constantes Globales
main.c	Llama a iniciarServidor
peticiones (.c y .h)	Manejan la lógica de recepción y redirección a controladores para procesamiento de peticiones. Luego, envían las respuestas correspondientes.
controladores comun (.c y .h) jugador (.c y .h) partida (.c y .h)	Leen los datos enviados por el cliente y manipulan los archivos para cumplir con las peticiones recibidas. Generan las respuestas (*)



ranking (.c y .h)	
(*) El flujo principal del programa pasa por estos archivos	

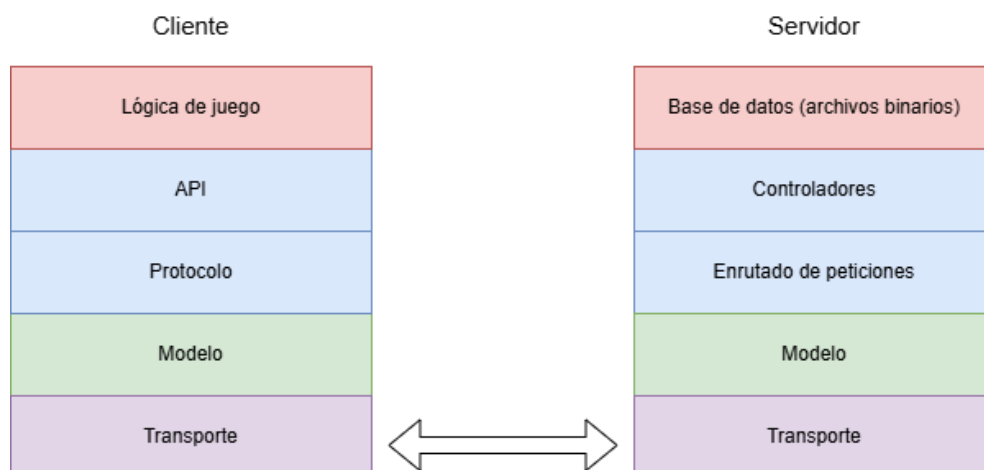
La separación en módulos permite al servidor trabajar sus responsabilidades en capas separadas (conexión, peticiones y controladores), pero interconectadas para poder cumplir sus funciones.

Comunicación cliente - servidor

Si bien tanto el cliente como el servidor tienen su sistema y separación en módulos, existe una serie de funciones y estructuras en común que facilitan la comunicación. Teniendo esto en cuenta, podemos identificar dos capas:

1. Capa de transporte: Utiliza un par de sockets TCP/IP, los cuales permiten el envío y recepción de secuencias de bytes sin pérdida.
2. Capa de modelo: Incluye una serie de “modelos” (Jugador, Partida, Ranking), estructuras en común que les permiten interpretar correctamente los datos a trabajar antes de convertirlos en secuencias de bytes a enviar, y de forma inversa, convertir al formato correcto los bytes recibidos. También incluye una serie de funciones utilitarias (leerCampo, cargarCampo) que les permiten trabajar con la capa de transporte de manera más sencilla, sin necesidad de hacer manipulaciones directas sobre las secuencias de bytes.

Viéndolo globalmente, podemos pensar tanto al cliente como al servidor como un sistema de capas, siendo en ambos casos, el más alto nivel lo que involucra una lógica menos relacionada con la red (la lógica de juego para el cliente o la base de datos para el servidor), hasta la capa más baja de socket, donde se produce la comunicación. Cada capa se comunica con las que tiene por encima y por debajo.

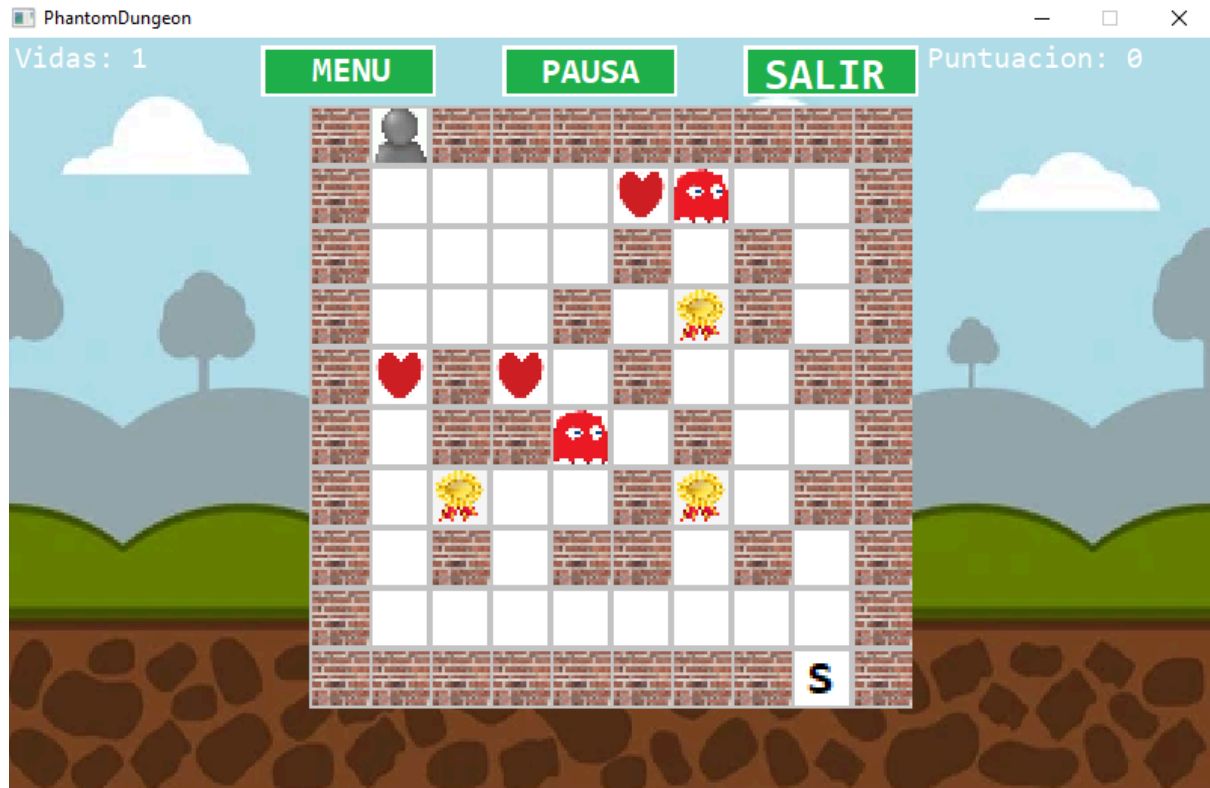


Una vez habiendo realizado esta aproximación a cómo está diseñado el programa, ya está listo para empezar a leer el código si así lo desea.



Funcionalidades añadidas

- Interfaz Gráfica con SDL: Se añadió una interfaz gráfica para que la interacción del usuario con el juego sea más amena.



- Puntaje extra por ganar partida: El jugador siempre obtendrá el puntaje que acumule con los premios que haya recolectado durante la partida. Si logra ganarla, además recibirá el doble de puntos de la cantidad de celdas que haya tenido el laberinto en el que haya jugado.
- Paginado: Para la correcta muestra de los rankings que se van obteniendo, hemos decidido aplicar un sistema de paginado (con límites y salteos de registros) que permita mostrarlos de forma prolija en la interfaz gráfica.



Algoritmos

Algoritmo de Generación de Laberinto

Pasos principales

Primero creamos una matriz del tamaño del laberinto llena de paredes, y luego realizamos diversas acciones para obtener un laberinto jugable y más entretenido.

La generación del laberinto la dividimos en los siguientes pasos principales:

1. Generar la entrada y el primer camino
2. Generar todos los caminos con el algoritmo DFS
3. Generar una galería al entrar al laberinto
4. Cortar algunas paredes para que haya más caminos
5. Insertar entidades en el mapa

Este algoritmo trabaja con una estructura de datos pila para poder llevar a cabo su función.

1) Generar la entrada y el primer camino

Se obtiene una celda de un borde aleatorio, y con ella también el primer camino liberado (que será la única celda adyacente no diagonal que esté dentro de los límites del laberinto que no sean borde). Esta primer celda se apila en la pila del algoritmo.

2) Generar todos los caminos con el algoritmo DFS

Aquí se desapila y se verifica en qué celdas adyacentes no diagonales podría haber caminos. Esta verificación se realiza viendo si la celda posee algún otro camino a su alrededor. En caso de que haya celdas con posibilidad de ser caminos se elige una de ellas de forma aleatoria (si existe una sola posibilidad lógicamente no será aleatoria), se convierte en camino y se apila. En caso de que ninguna celda adyacente no diagonal fuera capaz de volverse camino entonces se desapila.

El resultado de esto es que el algoritmo genera un camino hasta que se encuentra con un "callejón sin salida", y en ese momento desanda el camino (desapilando) hasta la anterior bifurcación posible.

Ejemplo Laberinto 10x10 - Estado Inicial									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

Ejemplo Laberinto 10x10 - Paso 1									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
E	.	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#



En las siguientes imágenes hay casos donde hay bifurcaciones posibles entonces se sigue liberando camino

Ejemplo Laberinto 10x10 - Paso 2 Hay bifurcaciones posibles									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
E	.	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

Ejemplo Laberinto 10x10 - Paso 2 Hay bifurcaciones posibles									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	.	.	.	#	#	#
#	#	#	#	.	#	#	#	#	#
E	.	.	.	.	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

En las siguientes imágenes se ve el caso donde no existen bifurcaciones posibles debido a que todas las paredes adyacentes también tienen como adyacente a otro camino. Cuando esto ocurre se “vuelve hacia atrás” hasta la anterior posible bifurcación. En la imagen hay números a modo ilustrativo de cómo se encuentran ubicadas esas celdas en la pila, siendo mayor número el tope y menor número la base. Aquí puede verse cómo se retrocederá hasta la celda “11” que es la anterior que puede hacer continuar el camino, y desde ella se retoma y se siguen liberando celdas.

Ejemplo Laberinto 10x10 - Paso 2 No hay bifurcaciones posibles									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	.	.	.	#	.	#
#	#	#	#	.	#	.	.	.	#
E	.	.	.	.	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

Ejemplo Laberinto 10x10 - Paso 2 No hay bifurcaciones posibles									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	14	13	#
#	#	#	#	6	7	8	#	12	#
#	#	#	#	5	#	9	10	11	#
E	1	2	3	4	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

Ejemplo Laberinto 10x10 - Paso 2 Vuelven a haber bifurcaciones posibles									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	.	.	#
#	#	#	#	6	7	8	#	.	#
#	#	#	#	5	#	9	10	11	#
E	1	2	3	4	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

Adicionalmente durante cada nuevo camino colocado, se preguntará si el camino es adyacente a una pared externa, y si lo es a la pared externa opuesta a la entrada, para entonces identificar una posible salida candidata. Al finalizar el algoritmo, la última salida candidata encontrada será la que se colocara en el mapa



Ejemplo posicionamiento de Salida Candidata									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	-	-	-	#	#	#
#	#	#	#	-	#	-	-	-	#
E	-	-	-	-	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

Ejemplo posicionamiento de Salida Candidata									
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	-	#
#	#	#	#	-	-	-	#	-	#
#	#	#	#	-	#	-	-	-	#
E	-	-	-	-	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

Ejemplo posicionamiento de Salida Candidata Final									
#	#	#	#	#	#	#	#	#	#
#	-	-	#	#	-	#	-	-	#
#	#	-	-	-	-	-	#	-	#
#	#	#	#	-	#	-	-	-	#
E	-	-	-	-	#	#	#	-	#
#	#	#	#	#	-	-	-	-	#
#	-	-	-	#	#	-	#	#	#
#	-	#	-	#	-	-	-	-	#
#	-	#	-	-	-	#	-	#	#
#	#	#	#	#	#	#	#	#	#

Posicionamiento de la Salida									
#	#	#	#	#	#	#	#	#	#
#	-	-	#	#	-	#	-	-	#
#	#	-	-	-	-	-	#	-	#
#	#	#	#	-	#	-	-	-	#
E	-	-	-	-	#	#	#	-	#
#	#	#	#	#	-	-	-	-	#
#	-	-	-	#	#	-	#	#	#
#	-	#	-	#	-	-	-	-	S
#	-	#	-	-	-	#	-	#	#
#	#	#	#	#	#	#	#	#	#

3) Ajustar los caminos del mapa

Luego del proceso anterior se obtendrá un mapa inicial. Sin embargo, todavía no es suficiente, ya que los caminos que genera son muy estrechos y no abren muchos caminos alternativos para que el jugador pueda escapar de los fantasmas. Entonces le aplicamos dos procesos.

El primer proceso que se le aplica genera una “galería” adyacente a la entrada, esencialmente un área de 3x3 que sea solo caminos. Esto incrementa la posibilidad de que el jugador encuentre más caminos alternativos para llegar a la salida, y que no pueda ser encerrado por los fantasmas al inicio de la partida.



Posicionamiento de la Galeria									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	.	#	.	.	#
#	#	.	.	.	.	.	.	#	.
#	.	.	.	.	#	.	.	.	#
E	.	.	.	.	#	#	#	.	#
#	.	.	.	#	.	.	.	.	#
#	.	.	.	#	#	.	#	#	#
#	.	#	.	#	.	.	.	.	S
#	.	#	.	.	.	#	.	#	#
#	#	#	#	#	#	#	#	#	#

El segundo proceso, aleatoriamente buscará en todo el mapa paredes que pueda “derrumbar”. Se consideran solo las paredes que tienen a ambos lados (horizontal o verticalmente) caminos, de forma que su destrucción garantiza la conexión de 2 caminos preexistentes. El aumento en la cantidad de caminos incrementa la capacidad de maniobra del jugador, y genera más rutas posibles hacia la salida.

Posibles cortes de paredes									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	.	#	.	.	#
#	#	.	.	.	.	.	#	.	#
#	.	.	.	.	#	.	.	.	#
E	.	.	.	.	#	.	#	.	#
#	.	.	.	#	.	.	.	.	#
#	.	.	.	#	#	.	#	.	#
#	.	#	.	.	.	.	.	.	S
#	.	#	.	.	.	#	.	#	#
#	#	#	#	#	#	#	#	#	#



4) Insertar entidades en el mapa

El último paso es posicionar las entidades en el mapa, siendo estas los fantasmas, los premios y las vidas extra. Lo que se hace es recorrer el mapa y encontrar todas las posiciones vacías (con caminos), se selecciona una de ellas de forma random, se elige, de todas las entidades a colocar una de ellas de forma random, y luego se coloca la entidad en la posición.

Mapa con una entidades posicionadas.									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	.	#	.	.	#
#	#	.	.	.	.	V	#	.	#
#	.	.	.	.	#	.	.	.	#
E	.	.	.	.	#	.	#	.	#
#	.	.	.	#	F	.	.	.	#
#	.	.	.	#	#	.	#	.	#
#	.	#	.	.	.	F	.	.	S
#	P	#	.	.	.	#	.	#	#
#	#	#	#	#	#	#	#	#	#

Una cosa que tenemos en cuenta al insertar entidades es en qué posiciones no se pueden insertar, particularmente, muy cerca de la entrada, y cerca de otra entidad. Entonces generamos un máscara que permitirá saber cuales son aquellos caminos bloqueados por proximidad a la salida u a otra entidad.



Mapa para comparación.									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	.	#	.	.	#
#	#	.	.	.	.	.	#	.	#
#	.	.	.	.	#	.	.	.	#
E	.	.	.	.	#	.	#	.	#
#	.	.	.	#	.	.	.	.	#
#	.	.	.	#	#	.	#	.	#
#	.	#	.	.	.	.	.	.	S
#	.	#	.	.	.	#	.	#	#
#	#	#	#	#	#	#	#	#	#

Mapa de mascara bloqueando caminos cercanos a la entrada									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	.	#	.	.	#
#	#	#	#	.	.	.	#	.	#
#	#	#	#	.	#	.	.	.	#
#	#	#	#	.	#	.	#	.	#
#	#	#	#	#	.	.	.	.	#
#	#	#	#	#	#	.	#	.	#
#	.	#	.	.	.	.	.	.	S
#	.	#	.	.	.	#	.	#	#
#	#	#	#	#	#	#	#	#	#

Mapa con una entidad.									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	.	#	.	.	#
#	#	.	.	.	.	.	#	.	#
#	.	.	.	.	#	.	.	.	#
E	.	.	.	.	#	.	#	.	#
#	.	.	.	#	.	.	.	.	#
#	.	.	.	#	#	.	#	.	#
#	.	#	.	.	.	F	.	.	S
#	.	#	.	.	.	#	.	#	#
#	#	#	#	#	#	#	#	#	#

Mapa de mascara bloqueando caminos cercanos a la entidad									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	.	#	.	.	#
#	#	#	#	.	.	.	#	.	#
#	#	#	#	.	#	.	.	.	#
#	#	#	#	.	#	.	#	.	#
#	#	#	#	#	.	.	.	.	#
#	#	#	#	#	#	#	#	.	#
#	.	#	.	.	#	#	#	.	S
#	.	#	.	.	#	#	#	#	#
#	#	#	#	#	#	#	#	#	#

El utilizar máscaras también favorece a la generación de laberintos jugables. Ya que de otra forma, las elecciones de laberintos de dimensiones pequeñas y altas cantidades de fantasmas vuelven imposible ganar la partida. Con este método evitamos que se generen demasiados fantasmas, ya que cuando la máscara cubra todo el mapa ya no habrá espacio para que se ubiquen más.



Algoritmo de Movimiento de Fantasmas

Para asegurar que los fantasmas constituyan un desafío significativo para el jugador, su comportamiento se modela mediante la Búsqueda en Amplitud (BFS). Este algoritmo de *pathfinding* les permite determinar invariablemente la ruta más corta hacia la posición actual del jugador. Este cálculo se reevalúa periódicamente para mantener una estrategia de persecución óptima.

Pasos Fundamentales del Rastreo Inteligente

El sistema de movimiento de cada fantasma opera en un ciclo compuesto por cinco etapas principales antes de ejecutar cualquier desplazamiento:

- Preparación de Herramientas
- Verificación de Velocidad (Cooldown)
- Ejecución del Rastreo (El "Radar" BFS)
- Descifrado del Camino
- Orden de Movimiento Final

1) Estructuras de memoria necesarias para el algoritmo

Antes de iniciar cualquier proceso lógico de movimiento, el fantasma establece sus herramientas de navegación:

- La Fila de Espera (Cola de Búsqueda): Este componente gestiona las celdas del mapa en un orden de exploración por "capas" o niveles de distancia. Esta estructura de datos garantiza que el primer camino hallado sea el más corto.
- El Mapa de Visitas: Una matriz auxiliar utilizada para registrar las celdas ya exploradas por el fantasma, evitando así ciclos infinitos y optimizando la velocidad de la búsqueda.
- El Mapa de Padres (Rastro de Precedencia): La función más crítica de la búsqueda. Cada vez que el fantasma visita una celda nueva, registra en esta matriz las coordenadas de la celda de origen o "padre". Esto establece un rastro inverso necesario para la reconstrucción de la ruta.
- El movimiento del fantasma está sujeto a un control temporal para modular su velocidad dentro del entorno de juego. Se emplea un sistema de "Cooldown" para prevenir movimientos excesivamente rápidos o erráticos.

2) Ejecución del Rastreo (El "Radar" BFS)

Esta fase constituye el núcleo central del algoritmo, donde el fantasma procede a determinar y localizar la ruta óptima hacia su objetivo.

El proceso se inicia definiendo el punto de partida como la ubicación actual del fantasma y la posición actual del jugador como el destino (la meta). A partir de ahí, el fantasma comienza una exploración sistemática inspeccionando las cuatro celdas adyacentes a su posición (los movimientos ortogonales: arriba, abajo, izquierda y derecha).



Para cada una de estas celdas vecinas, se aplican tres reglas de movimiento críticas que determinan si son candidatas viables para la ruta:

1. La celda debe estar dentro de los límites físicos del mapa.
2. No debe haber sido visitada previamente durante esta búsqueda (lo cual se gestiona eficientemente con una estructura de datos, como un Mapa de Visitas).
3. No debe representar una barrera física o un obstáculo, como una pared.

Las celdas que cumplen satisfactoriamente estas tres condiciones son encoladas en la Fila de Espera para ser analizadas en el siguiente ciclo. La búsqueda se detiene inmediatamente en el momento en que se encuentra la posición del jugador, marcando el fin de la exploración.

Este mecanismo de rastreo se visualiza como una expansión progresiva, similar a círculos concéntricos, que se origina en el fantasma (F) y se extiende por capas (1, 2, 3...) hasta que finalmente intercede la ubicación del jugador (J).

Ejemplo Laberinto 10x10 - Primeros pasos del algoritmo bfs									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	#	#	.	.	#
#	#	.	J	.	.	.	#	.	#
#	#	#	#	.	#	.	.	.	#
E	.	.	.	.	#	#	.	.	#
#	#	#	#	#	#	.	.	.	S
#	.	#	F	#	.	.	#	#	#
#	.	#	1	#	.	#	#	.	#
#	.	.	2	.	.	.	.	.	#
#	#	#	#	#	#	#	#	#	#

Ejemplo Laberinto 10x10 - División de caminos posibles									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	#	#	.	.	#
#	#	.	J	.	.	.	#	.	#
#	#	#	#	.	#	.	.	.	#
E	.	.	.	.	#	#	#	.	#
#	#	#	#	#	#	.	.	.	S
#	.	#	F	#	.	.	#	#	#
#	5	#	1	#	5	#	#	.	#
#	4	3	2	3	4	5	.	.	#
#	#	#	#	#	#	#	#	#	#

Ejemplo Laberinto 10x10 - encuentra el objetivo									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	#	#	15	14	#
#	#	.	J	17	16	15	#	13	#
#	#	#	#	.	#	14	13	12	#
E	.	.	.	.	#	#	#	11	#
#	#	#	#	#	#	8	9	10	S
#	6	#	F	#	6	7	#	#	#
#	5	#	1	#	5	#	#	8	#
#	4	3	2	3	4	5	6	7	#
#	#	#	#	#	#	#	#	#	#

3) Descifrado del Camino (Rastreo Inverso o Backtracking)

Una vez que la meta es localizada, el fantasma debe interpretar el camino de vuelta al origen para determinar su primer paso.

- Inicio del Rastro: El algoritmo se posiciona en la celda del jugador (la meta).
- Siguiendo la Cadena de Padres: En cada iteración, se consulta el Mapa de Padres para identificar la celda "padre" y retroceder a esa posición.
- El Primer Paso: Este rastreo se repite hasta que la celda de destino es la posición inicial del fantasma. El nodo guardado justo antes de este último salto (anterior) es, por lo tanto, la coordenada del primer movimiento óptimo a realizar.
- Caso Sin Camino: Si el proceso BFS no logra localizar al jugador, se devuelve una coordenada centinela que indica la ausencia de un camino viable.

Ejemplo Laberinto 10x10 - retrocede									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	#	#	15	14	#
#	#	.	J	17	16	15	#	13	#
#	#	#	#	.	#	14	13	12	#
E	.	.	.	.	#	#	#	11	#
#	#	#	#	#	#	8	9	10	S
#	6	#	F	#	6	7	#	#	#
#	5	#	1	#	5	#	#	8	#
#	4	3	2	3	4	5	6	7	#
#	#	#	#	#	#	#	#	#	#

Ejemplo Laberinto 10x10 - hasta encontrar el primer paso!									
#	#	#	#	#	#	#	#	#	#
#	.	.	#	#	#	#	15	14	#
#	#	.	J	17	16	15	#	13	#
#	#	#	#	.	#	14	13	12	#
E	.	.	.	.	#	#	#	11	#
#	#	#	#	#	#	8	9	10	S
#	6	#	F	#	6	7	#	#	#
#	5	#	1	#	5	#	#	8	#
#	4	3	2	3	4	5	6	7	#
#	#	#	#	#	#	#	#	#	#



La fase de cálculo culmina con la generación y envío de la instrucción de movimiento al motor de juego.

- Verificación de Destino: Se confirma que el primer paso encontrado es una celda por la que se puede transitar (no una pared ni una zona prohibida).
- Envío del Comando: Se construye una estructura Movimiento con el identificador del fantasma y las coordenadas del paso óptimo. Este comando se encola en la Fila Global de Movimientos, quedando pendiente para ser ejecutado por el bucle principal del juego.